

Drone-Based Delivery Optimization Using Exact and Metaheuristic Methods

Hamza Faiz Ahmed Fouatih Youcef Belaib Ishak Dib
Dhiaa Eddine Zeroual

National School of Artificial Intelligence (ENSIA)
Third Year – Numerical Methods and Optimisation

May 19, 2026

Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Complexity Analysis
- 4 Branch and Bound
- 5 Genetic Algorithm
- 6 Simulated Annealing
- 7 Experimental Results

Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Complexity Analysis
- 4 Branch and Bound
- 5 Genetic Algorithm
- 6 Simulated Annealing
- 7 Experimental Results

Context & Motivation

Why Drone Delivery?

- Rapid growth of e-commerce
- Traffic congestion in urban areas
- Need for fast, flexible last-mile logistics
- Companies: Amazon, DHL...

Key Operational Challenges

- Limited battery $\Rightarrow$ range constraint
- Limited payload $\Rightarrow$ weight constraint
- No-fly zones $\Rightarrow$ forbidden paths
- Fleet coordination



Problem Overview

Setting

- K drones, one central **depot** (node 0), n customers
- Each drone: depot $\rightarrow$ customers $\rightarrow$ depot
- Each customer has a **demand** q_i (weight)
- Arc costs d_{ij} = Euclidean distance (energy proxy)

Objective

$$\min \sum_{k=1}^K \text{cost}(r_k) = \min \text{ total energy consumed by all drones}$$

Constraints

- (C1) Each customer served *exactly once*
- (C2) Payload per route $\leq Q$ kg
- (C3) Distance per route $\leq B$ (battery)
- (C4) No traversal of forbidden arcs (no-fly zones)

Outline

- 1 Introduction
- 2 Mathematical Formulations**
- 3 Complexity Analysis
- 4 Branch and Bound
- 5 Genetic Algorithm
- 6 Simulated Annealing
- 7 Experimental Results

Formulation 1 — Arc-Flow MILP

Decision Variables

$x_{ijk} \in \{0, 1\}$: drone k travels arc $(i \rightarrow j)$ $u_{ik} \geq 0$: load of drone k at node i

Objective & Key Constraints

$$\min \sum_k \sum_{(i,j) \in A} d_{ij} \cdot x_{ijk}$$

$$\sum_k \sum_j x_{ijk} = 1 \quad \forall i \in \{1..n\} \quad \text{(coverage)}$$

$$\sum_j x_{ijk} = \sum_j x_{jik} \quad \text{(flow conservation)}$$

$$\sum_i q_i \sum_j x_{ijk} \leq Q \quad \text{(payload)}$$

$$\sum_{(i,j)} d_{ij} x_{ijk} \leq B \quad \text{(battery)}$$

$$u_{ik} - u_{jk} + Qx_{ijk} \leq Q - q_j \quad \text{(MTZ subtours)}$$

Formulation 2 — Set Partitioning / Flow Model

Idea

Enumerate all *feasible routes* $\mathcal{R}$ upfront. Select a subset covering all customers.

Variable: $y_r \in \{0, 1\}$ — route r is selected

$$\min \sum_{r \in \mathcal{R}} c_r \cdot y_r \quad \text{s.t.} \quad \sum_r a_{ir} y_r = 1 \quad \forall i, \quad \sum_r y_r \leq K$$

Formulation 1 (Arc-Flow)

- $O(n^2 K)$ variables
- Explicit MTZ subtour cuts
- Arc-level reasoning
- Natural for B&B

Formulation 2 (Set Partition)

- $O(|\mathcal{R}|)$ variables
- No subtour constraints
- Route-level reasoning
- Natural for column generation

Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Complexity Analysis**
- 4 Branch and Bound
- 5 Genetic Algorithm
- 6 Simulated Annealing
- 7 Experimental Results

Theorem

*The Drone-Based Delivery Optimization Problem is **NP-Hard**.*

Proof — see the report.

Practical Consequence: Exact methods are only tractable for small n ($\lesssim 12$). Metaheuristics are necessary for real-world sizes.



Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Complexity Analysis
- 4 Branch and Bound**
- 5 Genetic Algorithm
- 6 Simulated Annealing
- 7 Experimental Results

B&B — State Representation

Each B&B Tree Node Stores

$$\text{Node} = \langle \underbrace{\text{routes}}_{\text{partial routes}}, \underbrace{\text{unassigned}}_{\text{remaining customers}}, \underbrace{\text{committed_cost}}_{\text{fixed arc costs}} \rangle$$

Example ($n = 5$)

routes	{[3, 1], [], [5]}
unassigned	{2, 4}
committed	47.3

Branching:

Select unassigned customer with **highest demand** (most constrained first), then branch over all drone route positions.

Pruning if:

- Payload violated
- Battery violated
- $\text{LB} \geq \text{best known cost}$

B&B — Lower Bound & Upper Bound

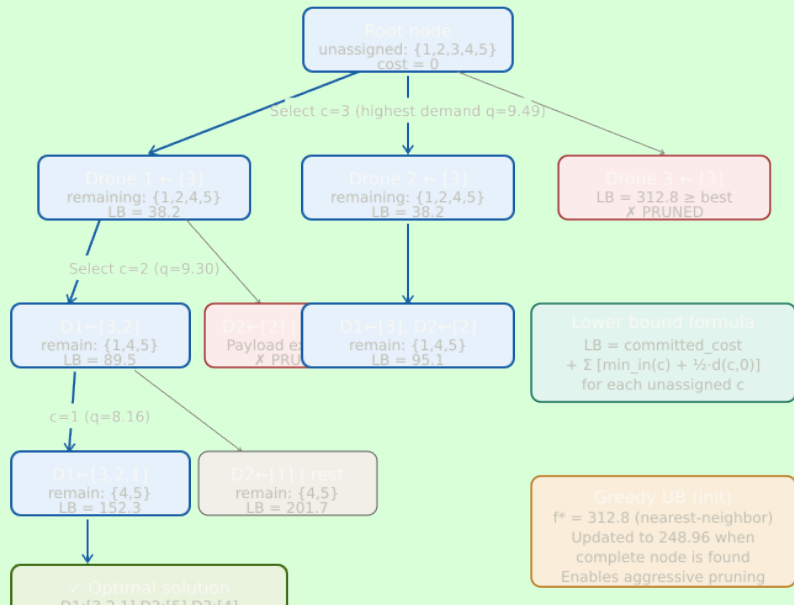
Lower Bound (Assignment Relaxation)

$$LB = committed_cost + \sum_{c \in unassigned} \left[\underbrace{\min_k d(\text{tail}(r_k), c)}_{\text{cheapest incoming arc}} + \underbrace{\frac{1}{2} d(c, 0)}_{\text{return share}} \right]$$

Upper Bound — Greedy Nearest-Neighbor

- 1 Start at depot; always go to the *closest feasible* unvisited customer.
- 2 Close route when payload or battery would be exceeded; start new drone.
- 3 Used to initialize f^* before B&B begins $\Rightarrow$ aggressive early pruning.

Optimal path
 Pruned ($LB \geq \text{best}$)
 Explored
 Complete (feasible)
 $n = 5$ customers



Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Complexity Analysis
- 4 Branch and Bound
- 5 Genetic Algorithm**
- 6 Simulated Annealing
- 7 Experimental Results

Giant Tour Permutation Encoding: $n = 6$ customers

$$\pi = [3, 1, 5, 2, 6, 4]$$

pos	1	2	3	4	5	6
π	3	1	5	2	6	4

Decoded into routes by greedy split (see next slide).

Greedy Split Procedure Decoding

Drone	Route	Reason closed
Drone 1	[3, 1, 5]	Payload full
Drone 2	[2, 6]	Battery limit
Drone 3	[4]	Last customer

Definition

The fitness of chromosome π is the **total energy cost** of its decoded solution:

$$f(\pi) = \sum_{r \in \text{decode}(\pi)} \text{cost}(r)$$

where

$$\text{cost}(r) = d(0, r_1) + \sum_{\ell=1}^{|r|-1} d(r_\ell, r_{\ell+1}) + d(r_{|r|}, 0)$$

Properties:

- **Lower is better**
- No penalty terms needed
- Feasibility guaranteed
- Evaluated once per individual

Tournament Selection ($k = 3$)

- 1 Sample 3 individuals uniformly at random from population
- 2 Select the one with **lowest fitness** (best cost)
- 3 Repeat independently to get two parents $\pi^{(1)}, \pi^{(2)}$

Invariant to fitness scaling; simple and efficient.

Order Crossover (OX) — rate $p_c = 0.85$

$$\pi^{(1)} = [3, 1, \mathbf{5, 2, 6}, 4] \quad (a=2, b=4)$$

$$\pi^{(2)} = [1, 6, 3, 4, 5, 2]$$

$$\text{child} = [_, _, \mathbf{5, 2, 6}, _] \xrightarrow{\text{fill}} [1, 3, 5, 2, 6, 4]$$

GA — Mutation & Local Search

Swap Mutation (rate p_m)

$$\pi_i \leftrightarrow \pi_j$$

Fine-grained perturbation.

Inversion Mutation (rate $0.7p_m$)

$$[\dots, A, B, C, \dots] \rightarrow [\dots, C, B, A, \dots]$$

Larger structural change.

Adaptive Mutation Rate

$$p_m(t) = p_{m,0} + 0.1 \cdot \frac{t_{\text{stuck}}}{T/3}$$

Increases during stagnation; capped at 0.5.

Or-opt Local Search (prob. 0.4)

Applied to each offspring:

- 1 Extract segment of 1–3 genes
- 2 Reinsert at another position
- 3 Accept if fitness improves
- 4 Repeat 20 times

Elitism

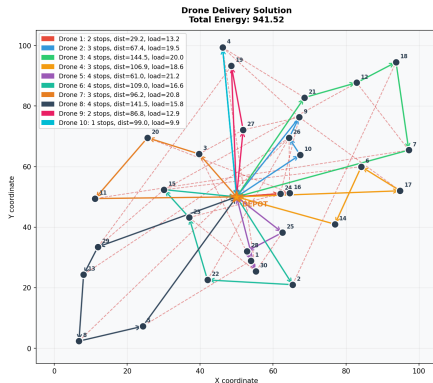
Top $e = 5$ chromosomes always survive to the next generation.

Best solution is never lost.

GA — Parameters & Convergence

Table: GA Parameters

Parameter	Value
Population size	80
Max generations	300
Crossover rate	0.85
Base mut. rate	0.15
Tournament k	3
Elite size	5
Or-opt iters	20



Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Complexity Analysis
- 4 Branch and Bound
- 5 Genetic Algorithm
- 6 Simulated Annealing**
- 7 Experimental Results

Solution Representation

Explicit list of drone routes:

$$S = \left[[3, 1, 5], [2, 6], [4] \right]$$

Initial Solution

Greedy nearest-neighbor: always pick the closest feasible unvisited customer; open new drone when capacity is exceeded.

SA — Neighborhood Operators (1/2)

Operator 1 — 2-opt Intra-Route

Reverse a sub-segment $[i..j]$ **within one drone's route**:

$$[\dots, r_i, r_{i+1}, \dots, r_j, \dots] \longrightarrow [\dots, r_j, r_{j-1}, \dots, r_i, \dots]$$

Operator 2 — Or-opt Relocation

Extract a contiguous segment of $s \in \{1, 2, 3\}$ customers from a **source route**, insert at a random position in a **destination route**:

$$r^{(k_1)} : [\dots, \underbrace{c_a, c_b}_{\text{segment}}, \dots] \rightarrow r^{(k_2)} : [\dots, c_a, c_b, \dots]$$

SA — Neighborhood Operators (2/2)

Operator 3 — Cross-Exchange (Swap)

Swap customer c_1 (from drone k_1) with customer c_2 (from drone k_2):

$$r^{(k_1)} : \dots, \mathbf{c}_1, \dots \leftrightarrow r^{(k_2)} : \dots, \mathbf{c}_2, \dots$$

Operator 4 — Route Merge

Concatenate two routes into one if the merged route is **feasible**:

$$r^{(k_1)} \parallel r^{(k_2)} = [r_1^{(k_1)}, \dots, r_1^{(k_2)}, \dots]$$

SA — Acceptance & Cooling

Metropolis Acceptance Criterion

$$P(\text{accept } S') = \begin{cases} 1 & \text{if } f(S') < f(S) \text{ (improvement)} \\ \exp\left(-\frac{f(S') - f(S)}{T}\right) & \text{otherwise (worsening)} \end{cases}$$

Geometric Cooling

$$T_{t+1} = \alpha \cdot T_t, \quad \alpha = 0.995$$

$$T_0 = 5 \times \bar{d} \quad (\text{auto-calibrated})$$

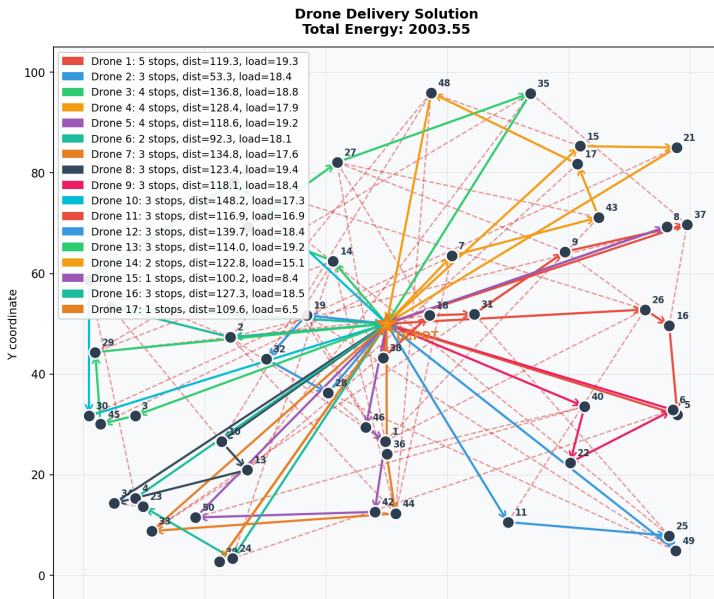
$\bar{d}$ = average arc length of instance.

Reheating (anti-stagnation)

If no improvement for $\tau = 500$ iterations:

$$T \leftarrow \min\left(1.5 T, \frac{T_0}{2}\right)$$

SA — Convergence



Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Complexity Analysis
- 4 Branch and Bound
- 5 Genetic Algorithm
- 6 Simulated Annealing
- 7 Experimental Results**

Benchmark Instances

Table: 10 Generated Instances

Instance	n	Q (kg)	B	No-fly
instance_01	5	24.92	155.8	1
instance_02	7	15.55	140.5	0
instance_03	8	14.22	174.4	0
instance_04	10	26.00	198.4	2
instance_05	12	18.90	189.0	7
instance_06	15	20.86	198.6	6
instance_07	20	19.29	182.8	12
instance_08	30	21.26	190.8	24
instance_09	50	19.41	194.0	48
instance_10	75	21.14	186.3	153

Customer positions: uniform in $[0, 100]^2$; demands: uniform in $[1, 10]$ kg.

Time limits: B&B = 8s, GA = 45s, SA = 45s.

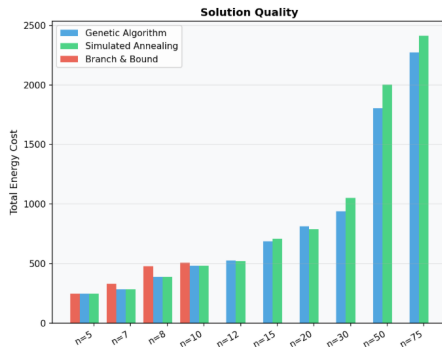
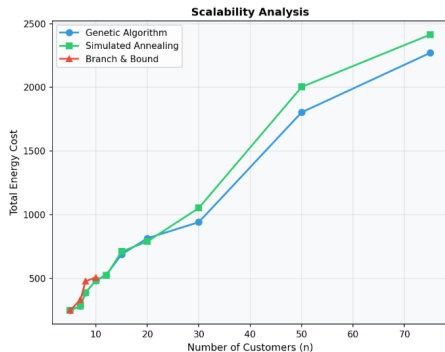
Results — Solution Quality

Table: Total Energy Cost by Method

Instance	n	B&B	GA	SA
instance_01	5	248.96	248.96	248.96
instance_02	7	332.92	283.86	283.86
instance_03	8	478.59	389.49	389.49
instance_04	10	506.78	483.52	483.52
instance_05	12	N/A	526.33	521.72
instance_06	15	N/A	688.32	711.10
instance_07	20	N/A	815.22	789.49
instance_08	30	N/A	941.52	1052.90
instance_09	50	N/A	1804.16	2003.55
instance_10	75	N/A	2271.19	2414.25

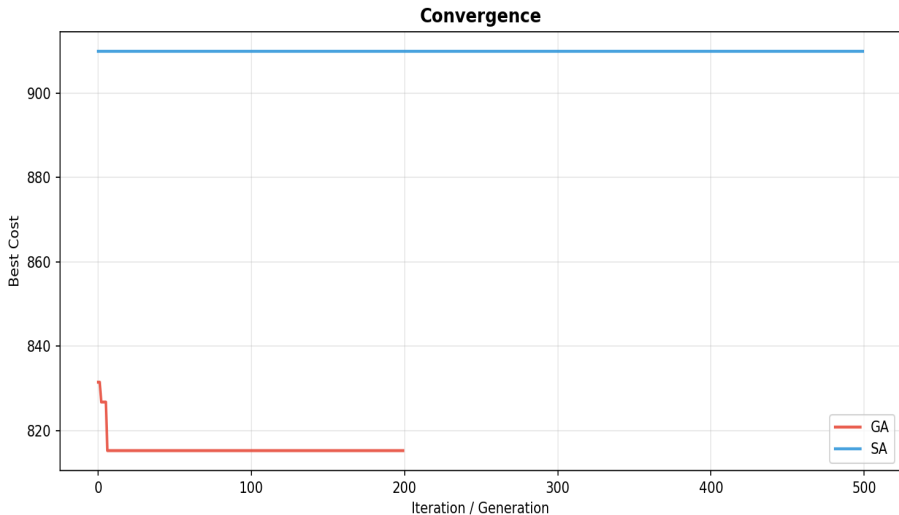
Bold = best for that instance. B&B not applied for $n > 12$.

Results — Charts



Both metaheuristics scale well. GA dominates SA for $n \geq 30$.

Results — Optimality Gap & Route Maps

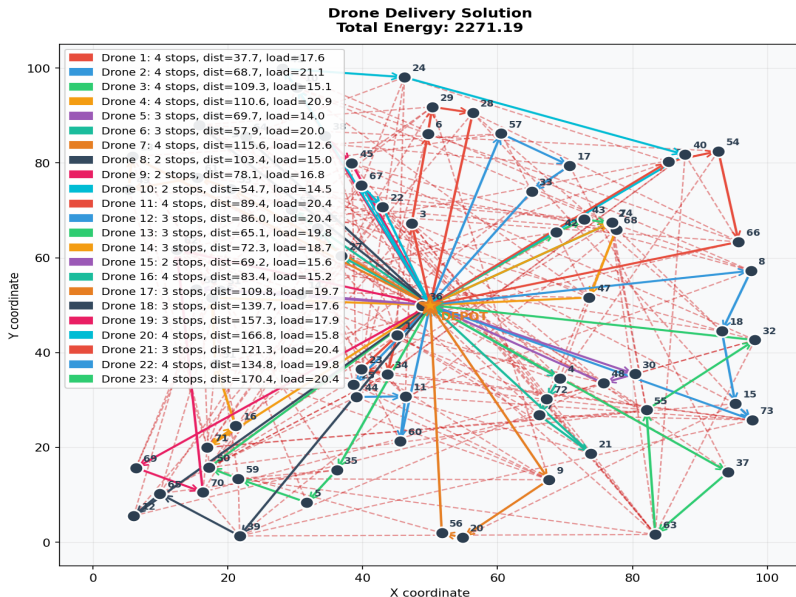


Comparative Analysis

Table: Method Comparison Summary

Criterion	B&B	GA	SA
Quality (small n)	Optimal	Near-optimal	Near-optimal
Quality (large n)	N/A	Best	Good
Time complexity	Exponential	$O(T \cdot P \cdot n)$	$O(\text{iter} \cdot n)$
Max tractable n	~ 12	75+	75+
Optimality guarantee	Yes	No	No
Implementation Tuning effort	Complex Low	Medium Medium	Simple Medium

Results — Optimality Gap & Route Maps



Recommendation

GA is the best overall method for real-world deployment due to superior scalability and solution quality on large instances. **SA** is a strong alternative when simplicity and speed are priorities.

Thank You

Questions?

Authors: Hamza Faiz Ahmed Fouatih
Youcef Belaib
Ishak Dib
Dhiaa Eddine Zeroual

Course: Numerical Methods and Optimisation
School: ENSIA – Third Year, 2026